



DateTime

🕒 Created time	July 25, 2026 5:52 AM
☰ Category	CommonVariables
👤 Last edited by	 R.A R.A
🕒 Last updated time	July 25, 2026 6:41 AM

```
using System.Globalization;

DateTime today = DateTime.UtcNow;

DateTime birthday = DateTime.Parse("15/06/1998");

DateTime birthday = DateTime.ParseExact("11/15/1998", "M/d/yyyy", CultureInfo.InvariantCulture);

DateTime birthday = DateTime.ParseExact("06/11/1998", "d/M/yyyy", CultureInfo.InvariantCulture);

Console.WriteLine(birthday.ToString());

Console.WriteLine(today.ToString("MMMM dd, yyyy hh:mm tt zz z", CultureInfo.InvariantCulture));
```

משתנה **DateTime** קיים כבר מתחילת ה-#C.

יש אפשרויות חדשות יותר (**חבילות**), אבל אובייקט ה-**DateTime** הוא הדרך הנפוצה ביותר לאחסן מידע על תאריך ושעה.

לדוגמה:

```
DateTime today = DateTime.Now;
```

זה יהיה התאריך והשעה של ממש עכשיו, וזה יוכנס למשתנה **today**.
זו שעה מקומית, בהתבסס על המיקום של המחשב שלנו.
יש הרבה מורכבות סביב תאריך ושעה.
הרעיון הוא שתאריך ושעה הם דבר מסובך, כי הם יכולים להיות שונים לכל אדם.
לדוגמה, אנחנו נמצאים באזור זמן **Central Time**, אז השעה **13:30** אצלנו היא בעצם **14:30 Eastern Time**, באותו הרגע בדיוק.
אז יש שם מורכבות של אזורי זמן.
יש גם דברים כמו שעון קיץ – האזור שלנו כן מקיים שעון קיץ, אבל אזורים אחרים, גם בתוך ארה"ב, לא מקיימים שעון קיץ.
ולכן, בהתאם לתקופה בשנה, יהיו שעות שונות.
אז יש הרבה מורכבות סביב תאריך ושעה – יש גם שנים מעוברות וכל שאר הדברים.
אבל אובייקט ה-**DateTime** מאחסן תאריך ושעה מלאים.
בדוגמה, אנחנו מדפיסים את זה:

```
Console.WriteLine(today);
```

ומאחורי הקלעים זה מדפיס כמחרוזת, כמו:

```
Console.WriteLine(today.ToString());
```

אבל כשמדפיסים את זה, יש לנו המון אפשרויות.
ל:

```
ToString()
```

הזה יש הרבה אפשרויות פורמט, כמו: **תאריך ארוך**, **תאריך/שעה מלא קצר**, **תאריך/שעה מלא ארוך**, **תאריך/שעה כללי קצר** וכן הלאה.
לדוגמה, אם אנחנו רוצים רק את התאריך – אות ה-**d הקטנה** – המשתנה עדיין מחזיק את אותו מידע; זה לא שינה את המידע שבמשתנה, אלא רק את מה שמוצג, מה שמודפס.
ואם אנחנו רוצים רק את השעה – **t קטנה**, זה בלי שניות; **T גדולה** זה עם שניות, וכן הלאה.
כך אפשר להשתמש בגרסאות השונות כדי להדפיס רק את החלק שרוצים.

במשך זמן רב ב-C# זה היה אובייקט **התאריך/שעה** היחיד שהיה קיים.

אז אם רוצים לייצג יום הולדת, נשתמש ב-**DateTime** בלי לדאוג לגבי חלק השעה, ובאופן מסורתי שמים את השעה כחצות – כך מטפלים בתאריך ב**לבד**.

ואם יש לנו רק **שעה** – למשל, שעת הפתיחה של עסק היא **08:00** – נייצג את התאריך כ-**1 בינואר, שנה 1**, ואת השעה כ-**08:00**.

זה קצת מסורבל, אבל אפשר לעשות את זה.

זו הדרך הנפוצה ביותר להתמודד עם תאריכים כשמשתמשים רק בחלק מהמידע, אבל יש גם אפשרויות ייעודיות של **תאריך בלבד ושעה בלבד** שיכולות להחליף את זה.

לדוגמה, יום הולדת:

```
DateTime birthday = DateTime.Parse("15/06/1998");  
Console.WriteLine(birthday.ToString());
```

זהו פורמט תאריך תקין – אבל תקין לפי ה-**locale** הישראלי, כי המחשב שלי מוגדר לישראל, וה-**locale** המקומי קובע **שהיום** מגיע ראשון, אחר כך **החודש**, ואז **השנה**.

אבל יש הרבה מקומות בעולם שבהם **החודש** מגיע ראשון והיום שני.

אז אם נריץ בדיוק את אותו קוד במחשב אחר, שבו ה-**locale** קובע **שהחודש** מגיע ראשון, הקוד יקרוס – כי אין **חודש 15**.

זו הסיבה ש**תאריך ושעה** יכולים להיות מסובכים מאוד: צריך לדעת את ה-**locale** שאיתו עובדים, ואם פשוט מניחים הנחות, אפשר להכניס באגים – ולפעמים באגים כאלה לא מקריסים את האפליקציה, אלא פשוט גורמים לה לפעול לא כמו שציפינו.

דרך אחרת לעשות את זה:

```
DateTime birthday = DateTime.ParseExact();
```

עם

ParseExact

אפשר להיות יותר מדויקים.

קודם נותנים לו את **התאריך**,

```
"06/11/1998"
```

ואז נותנים לו פורמט – עם אפשרויות מרשימת הפורמטים אפשר לציין מחרוזת מותאמת אישית. למשל, **d** – יום בחודש (1-2 ספרות): אם החודש הוא פברואר, נכתוב 2. אם בוחרים **dd** – יום בחודש (2 ספרות) – נכתוב 02.

אם נבחר:

```
DateTime birthday = DateTime.ParseExact("06/11/1998", "d/M/yyyy");
```

הפורמט

"d/M/yyyy"

נפוץ יותר בעולם (לא בארה"ב), והוא יפרש את "06/11/1998" כ-6 בנובמבר 1998.

d – יום קצר

M – גדולה – חודש

yyyy – שנה (4 ספרות)

וצריך גם לציין

CultureInfo.InvariantCulture

על ידי הוספת

```
using System.Globalization;
```

```
DateTime birthday = DateTime.ParseExact("06/11/1998", "d/M/yyyy", CultureInfo.InvariantCulture);
```

כלומר – "אל תתחשב באיזה **culture** או **locale** אתה נמצא, ואיך שם מפרשים פורמטים".

דוגמה נוספת:

```
Console.WriteLine(today.ToString("MMMM dd, yyyy hh:mm tt zz
```

```
z", CultureInfo.InvariantCulture));
```

שם החודש המלא – MMMM

יום ב-2 ספרות – dd

שנה ב-4 ספרות – yyyy

שעון 12 שעות (שעה ודקה, 2 ספרות כל אחד) – hh:mm

tt – AM/PM

zzz – UTC (שעות ודקות) הפרש אזור הזמן מ – zzz

עוד דוגמה:

```
DateTime today = DateTime.UtcNow;
```

עד עכשיו השתמשתי ב- **Now**, שזו השעה המקומית.

צריך לחשוב איפה הקוד באמת ירוץ, כי לרוב אנשים חושבים שאפשר להשתמש ב- **Now** כי הקוד תמיד ירוץ במקום שבו הם נמצאים – ובכן... אולי לא.

כי אם מעלים את האפליקציה לענן, היא עלולה לרוץ באזור זמן אחר, כי השרת עשוי להיות במיקום שונה.

אפשר לנסות לוודא שהשרת נמצא באותו אזור זמן שלנו, כך ש- **Now** עדיין יעבוד – אבל מה קורה כשהשרת נופל? כנראה יש גיבוי במקום אחר, ולרוב הגיבוי הזה נמצא באזור זמן שונה.

אז אם מתעדים לוגים על מתי דברים קרו באמצעות

```
DateTime.Now
```

בלי לשמור את ההפרש (**offset**) – זו עלולה להיות בעיה.

לכן יש אפשרות נוספת:

DateTime.UtcNow

זה זמן אוניברסלי מתואם (**UTC**) – מה שנקרא **זמן גריניץ'**, הזמן שכל שאר הזמנים מבוססים עליו.

אפשר לתעד הכול בזמן **UTC**, כי אז לא משנה אם הקוד רץ בחוף המזרחי, המערבי או באירופה – זה תמיד יהיה אותו זמן ביחס לכל שאר הזמנים.

עם זאת, מאבדים את היכולת לדעת מה היה ההפרש בזמן שהרשומה נוצרה, וצריך להחליט אם זה מקובל או לא.

אז יש זמן **UTC** וזמן מקומי – ומאבדים מידע כשלווקחים רק **UTC**, אבל בתמורה קל יותר להשוות בין תאריכים.